Reviewer Pack — Minimal p-Adic Valuation Degree and Communication Complexity…

Entry e16db217e1fb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 22:53:00 UTC

Minimal p-Adic Valuation Degree and Communication Complexity Rank

Entry ID: `e16db217e1fb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 22:53:00 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Valuation Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given Boolean function f, the minimal p-adic valuation degree (vd_p(f)) of its associated matrix representation is linearly correlated with its communication complexity rank (cr_f), such that vd_p(f) = Θ(cr_f).

Rationale (proposer's reasoning):

> p-adic analysis provides a framework for studying properties of number systems using non-archimedean valuations, which can potentially reveal hidden structures in the matrix representations of Boolean functions. This conjecture aims to connect these valuation properties with communication complexity, an area that studies the amount of communication needed between two parties to solve certain computational problems.

Taxonomy category: `CommunicationComplexityRank` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `aea19f16608a065c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> A conjecture is supported if the average correlation coefficient between the minimal p-adic valuation degree (vd_p(f)) and communication complexity rank (cr_f) of n-bit Boolean functions, computed over 30 random seeds, exceeds 0.7. A conjecture is falsified if this correlation coefficient is less than or equal to 0.3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (2): - `"p-adic valuation degree" AND "communication complexity rank"` - `"matrix representation" IN p-adic analysis AND communication complexity", "Boolean function" AND valuation theory AND matrix rank`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/2201.08874v2] Note on p-adic Local Functional Equation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def matrix_representation(f, n):
        A = []
        for i in range(2**n):
            row = []
            for j in range(2**n):
                x = bin(i)[2:].zfill(n)
                y = bin(j)[2:].zfill(n)
                z = ''.join(str(int(x[k]) ^ int(y[k])) for k in range(n))
                row.append(f[int(z, 2)])
            A.append(row)
        return A

    def p_adic_valuation_degree(A, p):
        n = len(A)
        max_val = 0
        for i in range(n):
            for j in range(n):
                if A[i][j] != 0:
                    val = 0
                    while A[i][j] % p == 0:
```

```
                    A[i][j] //= p
                    val += 1
                max_val = max(max_val, val)
        return max_val

    def communication_complexity_rank(A):
        n = len(A)
        rank = 0
        for i in range(n):
            if any(A[j][i] != 0 for j in range(n)):
                rank += 1
        return rank

    p = 2  # Fixed prime for p-adic valuation
    n_values = [5, 10, 15, 20, 30, 40]
    total_ratio = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            f = generate_boolean_function(n)
            A = matrix_representation(f, n)
            vd_p_f = p_adic_valuation_degree(A, p)
            cr_f = communication_complexity_rank(A)
            if vd_p_f != 0 and cr_f != 0:
                ratio = vd_p_f / cr_f
                total_ratio += ratio
                instances_tested += 1
                n_max = max(n_max, n)

    if instances_tested == 0:
        return {
            "metric_name": "vd_p(f) / cr_f",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }

    avg_ratio = total_ratio / instances_tested
    return {
        "metric_name": "vd_p(f) / cr_f",
        "metric_value": avg_ratio,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": 0.7 < avg_ratio <= 0.3,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - avg_metric_value)**2 for r in results if
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std={std_metric_value} support_fraction=
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std={std_metric_value} support_fraction=
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its execution to compute the required correlation coefficient. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12887
2	propose	ollama_remote	glm4:latest	0	0	13811
3	propose	ollama_remote	glm4:latest	0	0	12573
4	propose	ollama_remote	glm4:latest	0	0	12312
5	preregistration	ollama_remote	glm4:latest	0	0	8966
6	novelty	ollama_remote	glm4:latest	0	0	9846
7	novelty	ollama_remote	glm4:latest	0	0	9248
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	118944
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10212
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11247

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48215
12	judge	ollama_remote	glm4:latest	0	0	52895

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 321158 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/e16db217e1fb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e16db217e1fb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e16db217e1fb.tar.gz` (if generated)